Automated framework for generating NDVI maps using Sentinel-2 data in Google Earth Engine (10 m resolution)

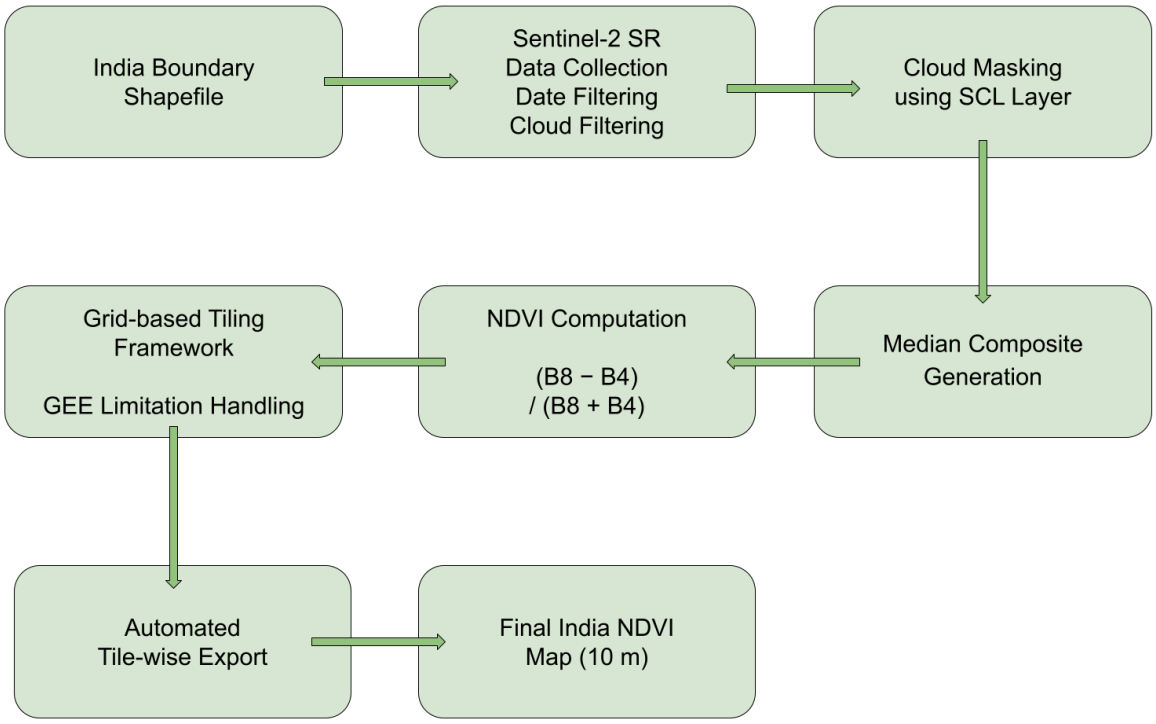
Brahmi Paliwal
9730013547
brahmipaliwal1@gmail.com
11 May 2026

Overview:

Sentinel-2 imagery was used in Google Earth Engine to generate an NDVI map of India at 10 m resolution. Cloud masking was applied using the SCL layer, followed by NDVI calculation using the B8 (NIR) and B4 (Red) bands. A grid-based tiling approach was used to get access to each part of the country on a deeper level.

Workflow Diagram:

The workflow below summarizes the methodology used for Sentinel-2 preprocessing, NDVI generation, tiling, and scalable export in Google Earth Engine.



Short Explanation:

The aim of this project was to develop an automated framework to generate NDVI (Normalized Difference Vegetation Index) maps for the whole of India using Sentinel-2 imagery in Google Earth Engine (GEE) at 10 m resolution.

Sentinel-2 Surface Reflectance imagery was used because it provides high-resolution satellite data that works well for vegetation analysis. The data was filtered for the period from January 2024 to March 2024, and images with more than 20% cloud cover were removed to improve the quality of the output.

To reduce cloud disturbance further, cloud masking was applied using the Scene Classification Layer (SCL) available in Sentinel-2 data. Cloud shadows, clouds, and cirrus pixels were removed before processing. This helped generate a cleaner NDVI output with less noise.

After preprocessing, a median composite image was created to reduce remaining cloud effects and combine multiple satellite images into one smoother image. NDVI was then calculated using the Near Infrared (B8) and Red (B4) bands. NDVI is commonly used to study vegetation, where higher values usually show healthier or denser vegetation, while lower values represent sparse vegetation or non-vegetated areas. NDVI values generally range from -1 to +1, where values closer to +1 indicate dense vegetation.

Since generating NDVI for the whole of India at 10 m resolution can be difficult due to Google Earth Engine processing and export limitations, a grid-based tiling approach was used. India was divided into smaller tiles so that processing and export could be done more smoothly instead of trying to process the entire country at once. This made the workflow more scalable and easier to handle for large datasets.

During implementation, a small technical issue came up because Sentinel-2 images sometimes contain different supporting bands across scenes. To avoid errors and keep the image collection consistent, only the required bands (B4, B8, and SCL) were selected before processing.

The final outputs include an India-wide NDVI map, a tiling-based processing framework, and regional close-up visualizations to show vegetation variation across different areas. The same workflow can also be extended further using the Python-GEE API for full automation and seasonal monitoring in the future.

Results and Visualizations:

Figure 1.1

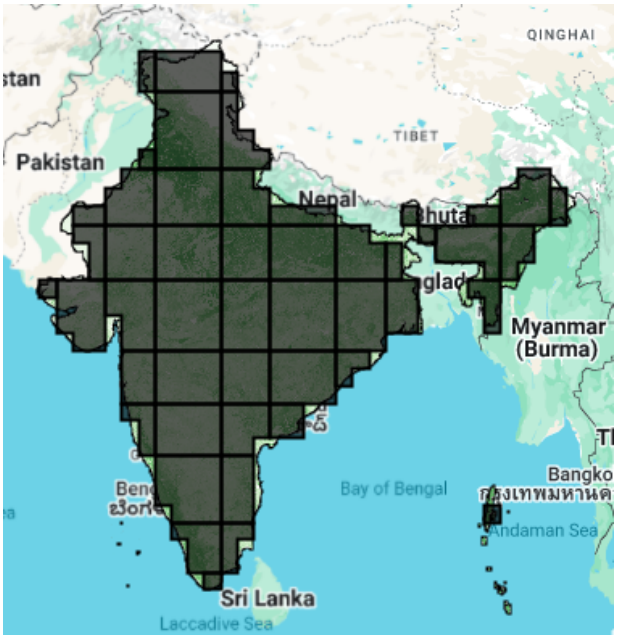


Figure 1.2

Figure 1.1–1.2: NDVI map of India generated using Sentinel-2 imagery in Google Earth Engine at 10 m resolution.
Figure 1.1 shows the final NDVI output, while **Figure 1.2** includes the grid-based tiling framework used for smoother processing and export across the country.

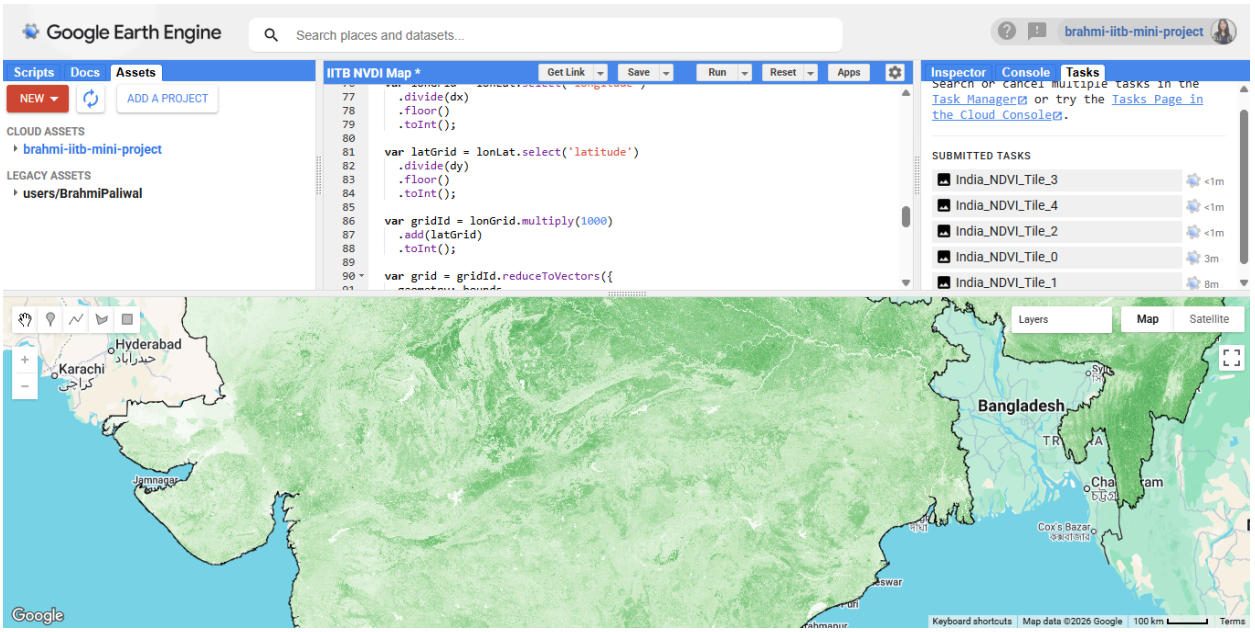


Figure 2.1

Figure 2.1: Google Earth Engine workspace showing the implemented NDVI workflow, code editor, map output, assets, and tile-wise export tasks used for scalable processing.

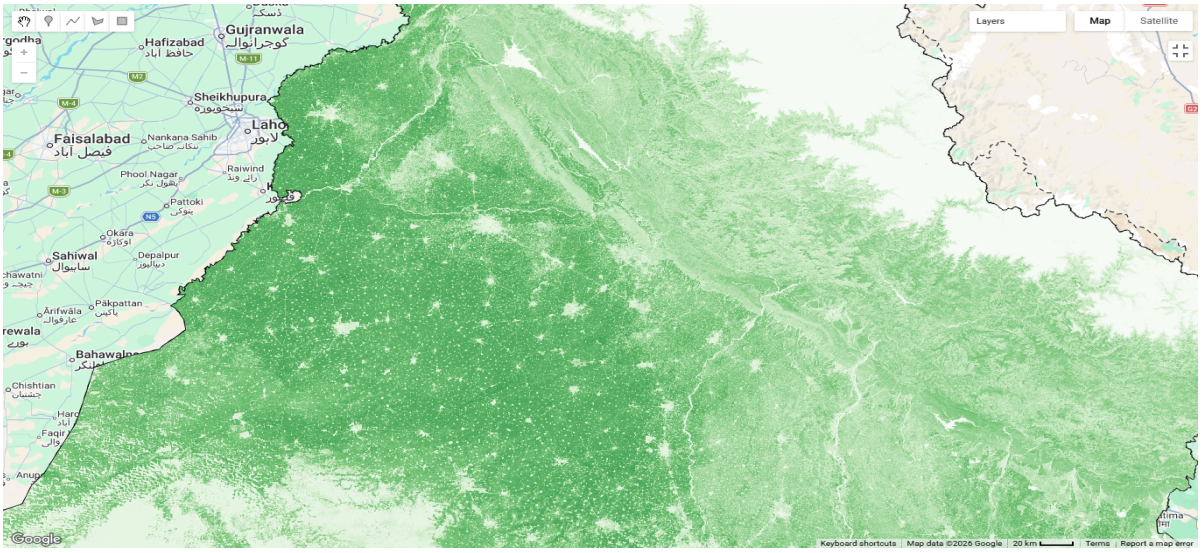


Figure 3.1

Figure 3.1: Close-up NDVI map of central and northern India, showing different vegetation patterns and the Himalayan region in the north.

Google Earth Engine (GEE) Script:

```
// 1. Map Initialization
Map.setCenter(78.9629, 22.5937, 4);

// 2. India Boundary
var india = ee.FeatureCollection("USDOS/LSIB_SIMPLE/2017")
  .filter(ee.Filter.eq('country_na', 'India'));

Map.addLayer(india, {color: 'black'}, 'India Boundary');

// 3. Sentinel-2 Data Collection
var s2 = ee.ImageCollection('COPERNICUS/S2_SR')
  .filterBounds(india)
  .filterDate('2024-01-01', '2024-03-31')
  .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 20))
  .select(['B4', 'B8', 'SCL']);
```

```
// 4. Cloud Masking using SCL Layer
function maskS2(image) {

  var scl = image.select('SCL');

  var mask = scl.neq(3)    // Cloud shadow
    .and(scl.neq(8))       // Cloud medium probability
    .and(scl.neq(9))       // Cloud high probability
    .and(scl.neq(10));     // Cirrus

  return image.updateMask(mask);--
}

// Apply cloud masking
var maskedS2 = s2.map(maskS2);

// 5. Median Composite and NDVI Calculation
var composite = maskedS2
  .select(['B4', 'B8'])
  .median()
  .clip(india);

var ndvi = composite
  .normalizedDifference(['B8', 'B4'])
  .rename('NDVI');

// 6. NDVI Visualization
var ndviParams = {
  min: 0,
  max: 1,
  palette: [
    '#f7fcf5',
    '#c7e9c0',
    '#74c476',
    '#238b45',
    '#00441b'
  ]
};

Map.addLayer(ndvi, ndviParams, 'India NDVI');

// 7. Grid-based Tiling for Scalable Processing
function createGrid(bounds, dx, dy) {

  var lonLat = ee.Image.pixelLonLat();

  var lonGrid = lonLat.select('longitude')
    .divide(dx)
    .floor()
    .toInt();

  var latGrid = lonLat.select('latitude')
```

```
    .divide(dy)
    .floor()
    .toInt();

  var gridId = lonGrid.multiply(1000)
    .add(latGrid)
    .toInt();

  return gridId.reduceToVectors({
    geometry: bounds,
    scale: 100000,
    geometryType: 'polygon',
    reducer: ee.Reducer.countEvery(),
    labelProperty: 'tile_id'
  });
}

// Generate processing grid
var grid = createGrid(india.geometry(), 3, 3);

Map.addLayer(grid, {color: 'black'}, 'Processing Tiles');

// 8. Tile-wise Export to Google Drive
var tileList = grid.toList(grid.size());
for (var i = 0; i < 5; i++) {

  var tile = ee.Feature(tileList.get(i));
  Export.image.toDrive({
    image: ndvi.clip(tile.geometry()),
    description: 'India_NDVI_Tile_' + i,
    folder: 'GEE_NDVI_India',
    region: tile.geometry(),
    scale: 10,
    maxPixels: 1e13
  });
}
```

Platform Used: Google Earth Engine (JavaScript API)